

NOPT061: Úvod, Heuristiky, Metaheuristiky

Výtah z 1. přednášky

Příprava na zkoušku

1 Úvod do kombinatorické optimalizace

Problém kombinatorické optimalizace (CO) je definován pomocí konečné množiny objektů $\mathcal{S}$ a účelové funkce $f : \mathcal{S} \mapsto \mathbb{R}^+$, která každému objektu přiřazuje nezápornou hodnotu. Proces optimalizace spočívá v nalezení objektu s^* s minimální hodnotou účelové funkce. Minimalizace funkce f je ekvivalentní maximalizaci funkce $-f$, proto lze každý CO problém popsat jako minimalizační problém.

Přesné (exaktní) algoritmy garantují nalezení optimálního řešení v konečném čase pro instance konečné velikosti. Za předpokladu, že $P \neq NP$, však algoritmus řešící NP-těžký problém v polynomiálním čase neexistuje. Proto mohou přesné metody v nejhorším případě vyžadovat exponenciální čas. Z tohoto důvodu se využívají aproximační metody, které obětují garanci nalezení optima za účelem produkce vysoce kvalitních řešení v prakticky přijatelném čase.

2 Základní heuristické metody

2.1 Konstruktivní heuristiky (Constructive Heuristics)

Předpokládáme, že řešení je složeno z množiny možných komponent $\mathcal{C}$. Platné řešení s je reprezentováno jako podmnožina $\mathcal{C}$, tedy $s \subseteq \mathcal{C}$. Konstruktivní heuristiky generují řešení "od nuly" tak, že v každém kroku přidávají komponentu do zpočátku prázdného částečného (parciálního) řešení s^p . Množina možných rozšíření pro částečné řešení se značí $Ext(s^p) \subseteq \mathcal{C}$.

Nejznámějším příkladem je chamtivá (greedy) heuristika, která využívá greedy funkci k ohodnocení kvality komponent a v každém kroku vybírá rozšíření s nejvyšší hodnotou (např. hranu s nejmenší váhou).

Algorithm 1 Constructive Heuristic

```
1:  $s^p \leftarrow \emptyset$   
2: while  $Ext(s^p) \neq \emptyset$  do  
3:    $c \leftarrow Select(Ext(s^p))$   
4:   přidej  $c$  do  $s^p$   
5: end while
```

2.2 Místní hledání (Local Search)

Místní hledání začíná z počátečního řešení a v každém kroku nahrazuje aktuální řešení lepším řešením z jeho vhodně definovaného okolí.

Definice 1 (Funkce okolí). Funkce okolí $\mathcal{N} : \mathcal{S} \rightarrow 2^{\mathcal{S}}$ přiřazuje každému řešení $s \in \mathcal{S}$ množinu sousedů $\mathcal{N}(s) \subseteq \mathcal{S}$, která se nazývá okolí s .

Definice 2 (Lokální minimum). Lokální minimum vzhledem k funkci okolí $\mathcal{N}$ je řešení $\hat{s}$ takové, že $\forall s \in \mathcal{N}(\hat{s}) : f(\hat{s}) \leq f(s)$. Řešení $\hat{s}$ nazýváme striktní lokální minimum, pokud platí ostrá nerovnost $f(\hat{s}) < f(s)$.

Existují dva hlavní přístupy pro výběr zlepšujícího souseda: *First improvement* (vrací první nalezené řešení, které je lepší než s) a *Best improvement* (prohledá celé okolí a vrátí nejlepší nalezené řešení).

Algorithm 2 Local Search

Vstup: počáteční řešení s , funkce okolí $\mathcal{N}$

- 1: **while** $\exists s' \in \mathcal{N}(s)$ takové, že $f(s') < f(s)$ **do**
 - 2: $s \leftarrow \text{ChooseImprovingNeighbor}(\mathcal{N}(s))$
 - 3: **end while**
-

2.3 Variable Neighborhood Descent (VND)

Rozšíření deterministického místního hledání, které využívá konečnou množinu více funkcí okolí $\{\mathcal{N}_1, \dots, \mathcal{N}_{k_{max}}\}$. Systematicky je prozkoumává, dokud není dosaženo řešení, které je lokálně optimální vůči všem těmto funkcím. Funkce jsou často seřazeny podle velikosti okolí: $|\mathcal{N}_1(s)| \leq |\mathcal{N}_2(s)| \leq \dots \leq |\mathcal{N}_{k_{max}}(s)|$.

Algorithm 3 Variable Neighborhood Descent (VND)

Vstup: počáteční řešení s , funkce okolí $\mathcal{N}_k, k = 1, \dots, k_{max}$

- 1: $k \leftarrow 1$
 - 2: **while** $k \leq k_{max}$ **do**
 - 3: $s' \leftarrow \text{ChooseBestNeighbor}(\mathcal{N}_k(s))$
 - 4: **if** $f(s') < f(s)$ **then**
 - 5: $s \leftarrow s'$
 - 6: $k \leftarrow 1$
 - 7: **else**
 - 8: $k \leftarrow k + 1$
 - 9: **end if**
 - 10: **end while**
-

3 Metaheuristiky

Metaheuristiky jsou obecné aproximační algoritmy, které kombinují konstruktivní heuristiky a místní hledání se strategiemi pro efektivní prohledávání stavového prostoru a především obsahují mechanismy pro únik z lokálních minim.

3.1 Greedy Randomized Adaptive Search Procedure (GRASP)

Kombinuje randomizovanou chamtivou konstrukci řešení s následnou aplikací místního hledání. Při konstrukci je z možných rozšíření $Ext(s^p)$ vytvořen tzv. *restricted candidate*

list L , obsahující nejlepší komponenty, ze kterých se následně volí náhodně. Parametr délky seznamu α řídí míru diverzity a heuristického biasu.

Algorithm 4 GRASP

```

1: while podmínky ukončení nejsou splněny do
2:    $s \leftarrow \text{ConstructGreedyRandomizedSolution}()$ 
3:    $s \leftarrow \text{LocalSearch}(s)$ 
4: end while

```

3.2 Iterated Greedy (IG)

V každé iteraci je aktuální řešení s částečně destruováno probabilistickým způsobem na částečné řešení s^p . Následně probíhá regenerační proces na kompletní řešení s' , na které může být (volitelně) aplikováno místní hledání.

Algorithm 5 Iterated Greedy (IG)

```

1:  $s \leftarrow \text{GenerateInitialSolution}()$ 
2: while podmínky ukončení nejsou splněny do
3:    $s^p \leftarrow \text{DestroyPartially}(s)$ 
4:    $s' \leftarrow \text{RecreateCompleteSolution}(s^p)$ 
5:    $\hat{s}' \leftarrow \text{LocalSearch}(s')$  {volitelné}
6:    $s \leftarrow \text{ApplyAcceptanceCriterion}(s, \hat{s}', \text{history})$ 
7: end while

```

3.3 Iterated Local Search (ILS)

Startovní řešení pro novou iteraci je generováno náhodnou perturbací aktuálního nejlepšího řešení. Perturbace by měla proces přesunout do jiné oblasti přitažlivosti (basin of attraction), ale měla by zůstat blízko aktuálnímu řešení.

Algorithm 6 Iterated Local Search (ILS)

```

1:  $s \leftarrow \text{GenerateInitialSolution}()$ 
2:  $s \leftarrow \text{LocalSearch}(s)$ 
3: while podmínky ukončení nejsou splněny do
4:    $s' \leftarrow \text{Perturbation}(s, \text{history})$ 
5:    $\hat{s}' \leftarrow \text{LocalSearch}(s')$ 
6:    $s \leftarrow \text{ApplyAcceptanceCriterion}(s, \hat{s}', \text{history})$ 
7: end while

```

3.4 Simulated Annealing (SA)

Metaheuristika inspirovaná procesem žhání kovů a skla. Dovoluje tahy k horším řešením (tzv. uphill moves) v závislosti na parametru teploty T_k a rozdílu $f(s') - f(s)$. Pravděpodobnost přijetí horšího řešení běžně klesá v čase a je počítána dle Boltzmannova rozdělení.

Algorithm 7 Simulated Annealing (SA)

```
1:  $s \leftarrow \text{GenerateInitialSolution}()$ 
2:  $k \leftarrow 0$ 
3:  $T_0 \leftarrow \text{SetInitialTemperature}()$ 
4: while podmínky ukončení nejsou splněny do
5:    $s' \leftarrow \text{SelectNeighborAtRandom}(\mathcal{N}(s))$ 
6:   if  $f(s') < f(s)$  then
7:      $s \leftarrow s'$ 
8:   else
9:     přijmi  $s'$  jako nové řešení s pravděpodobností  $p(s'|T_k, s)$ 
10:  end if
11:   $T_{k+1} \leftarrow \text{AdaptTemperature}(T_k, k)$ 
12:   $k \leftarrow k + 1$ 
13: end while
```

3.5 Tabu Search (TS)

Využívá krátkodobou paměť pro únik z lokálních minim a zamezení cyklení. Paměť je implementována jako tabu list (TL) omezené délky (tabu tenure), který uchovává historii nejnověji navštívených řešení a tím je omezuje v okolí. Vzniká tak restriktivní okolí $\mathcal{N}_a(s) = \mathcal{N}(s) \setminus TL$.

Algorithm 8 Tabu Search (TS)

```
1:  $s \leftarrow \text{GenerateInitialSolution}()$ 
2:  $TL \leftarrow \emptyset$ 
3: while podmínky ukončení nejsou splněny do
4:    $\mathcal{N}_a(s) \leftarrow \mathcal{N}(s) \setminus TL$ 
5:    $s' \leftarrow \arg \min\{f(s'') | s'' \in \mathcal{N}_a(s)\}$ 
6:    $\text{Update}(TL, s, s')$ 
7:    $s \leftarrow s'$ 
8: end while
```

3.6 Variable Neighborhood Search (VNS)

Pravděpodobnostní rozšíření deterministické metody VND, které mění funkce okolí pro diverzifikaci a únik z lokálních minim. Každá iterace obsahuje fázi "shaking" (náhodný výběr řešení z k-tého okolí), lokální prohledávání a posun.

Algorithm 9 Variable Neighborhood Search (VNS)

Vstup: množina funkcí okolí $\mathcal{N}_k, k = 1, \dots, k_{max}$

- 1: $s \leftarrow \text{GenerateInitialSolution}()$
- 2: **while** podmínky ukončení nejsou splněny **do**
- 3: $k \leftarrow 1$
- 4: **while** $k < k_{max}$ **do**
- 5: $s' \leftarrow \text{PickAtRandom}(\mathcal{N}_k(s))$ {shaking phase}
- 6: $s'' \leftarrow \text{LocalSearch}(s')$
- 7: **if** $f(s'') < f(s)$ **then**
- 8: $s \leftarrow s''$
- 9: $k \leftarrow 1$
- 10: **else**
- 11: $k \leftarrow k + 1$
- 12: **end if**
- 13: **end while**
- 14: **end while**

3.7 Ant Colony Optimization (ACO)

Inspirováno chováním mravenčích kolonií hledajících nejkratší cestu. Využívá parametrizovaný pravděpodobnostní feromonový model τ_i , kde jsou feromony asociovány s komponentami řešení. Zpětná vazba ve formě updatu feromonů zesiluje komponenty obsažené v kvalitních řešeních.

Algorithm 10 Ant Colony Optimization (ACO)

- 1: **while** podmínky ukončení nejsou splněny **do**
- 2: **Schedule Activities**
- 3: $\text{AntBasedSolutionConstruction}()$
- 4: $\text{PheromoneUpdate}()$
- 5: $\text{DaemonActions}()$ {volitelné}
- 6: **end Schedule Activities**
- 7: **end while**

3.8 Evolutionary Algorithms (EA)

Zjednodušené výpočetní modely evolučních procesů. Základem je populace jedinců P . Během každé iterace se volí podmnožina jedinců P^s (selekce na základě fitness), probíhá křížení/rekombinace za vzniku potomků P^{off} a následná případná mutace na množinu P' .

Algorithm 11 Evolutionary Algorithm (EA)

- 1: $P \leftarrow \text{GenerateInitialPopulation}()$
- 2: **while** podmínky ukončení nejsou splněny **do**
- 3: $P^s \leftarrow \text{Selection}(P)$
- 4: $P^{off} \leftarrow \text{Recombination}(P^s)$
- 5: $P' \leftarrow \text{Mutation}(P^{off})$
- 6: $P \leftarrow \text{Replacement}(P, P')$
- 7: **end while**

4 Přednáška 2 - Evoluční a memetické algoritmy, Exaktní metody

4.1 Evoluční a memetické algoritmy

4.1.1 Evoluční algoritmy (Evolutionary Algorithms - EA)

Evoluční algoritmy jsou silně zjednodušené výpočetní modely evolučních procesů inspirované přírodou a principem přežití nejsilnějšího jedince. Základem je populace P (množina) jedinců, kteří představují kandidátní řešení optimalizačního problému. V každé iteraci se na základě tzv. fitness (hodnoty účelové funkce) vybere podmnožina jedinců P^s , kteří slouží jako rodiče. Z nich se pomocí operátoru křížení (recombination/crossover) vytvoří potomci P^{off} . Následně se aplikuje operátor mutace, který způsobí drobné modifikace jedinců a vytvoří množinu P' . Nakonec se vybere populace pro další generaci pomocí operátoru nahrazení (replacement) z původní populace P a potomků P' .

Algorithm 12 Evolutionary Algorithm (EA)

```
1:  $P \leftarrow \text{GenerateInitialPopulation}()$ 
2: while podmínky ukončení nejsou splněny do
3:    $P^s \leftarrow \text{Selection}(P)$ 
4:    $P^{off} \leftarrow \text{Recombination}(P^s)$ 
5:    $P' \leftarrow \text{Mutation}(P^{off})$ 
6:    $P \leftarrow \text{Replacement}(P, P')$ 
7: end while
```

4.1.2 Memetické algoritmy (Memetic Algorithms - MA)

Memetické algoritmy jsou rozšířením genetických a evolučních algoritmů, které kombinují populační přístup s procedurou učení jednotlivce (místní prohledávání/local search). Začleněním heuristik lokálního prohledávání do stochastického globálního hledání pomáhají memetické algoritmy snižovat pravděpodobnost předčasné konvergence. Zatímco křížení umožňuje skoky napříč prostorem (globální hledání) a mutace prozkoumává lokální okolí, místní prohledávání (LSP) prohledává malý prostor blízko aktuálního řešení a nahrazuje jej lepším. Memetické algoritmy tak využívají specifické znalosti o problému.

Algorithm 13 Population-Based Search Engine (Základní šablona MA)

```
1: Inicializuj  $pop$  pomocí  $\text{GenerateInitialPopulation}()$ 
2: repeat
3:    $newpop \leftarrow \text{GenerateNewPopulation}(pop)$ 
4:    $pop \leftarrow \text{UpdatePopulation}(pop, newpop)$ 
5:   if  $pop$  konvergovala then
6:      $pop \leftarrow \text{RestartPopulation}(pop)$ 
7:   end if
8: until  $\text{TerminationCriterion}()$ 
```

Algorithm 14 Local search procedure (Procedura místního hledání v MA)

```
1: Funkce  $LSP(LLS, LS_r, LS_t, F, D, R, genotype)$ 
2: for  $i = 0$  to  $LLS - 1$  do
3:    $new\_solution \leftarrow Mutation(LS_r, LS_t, F, D, R, genotype)$ 
4:   if  $F(new\_solution) < F(genotype)$  then
5:      $genotype \leftarrow new\_solution$ 
6:   end if
7: end for
8:  $phenotype \leftarrow genotype$ 
9: return  $phenotype$ 
```

4.1.3 Lamarckovské vs. Baldwinovské učení

V kontextu memetických algoritmů se rozlišuje genotyp (genetická výbava jednotlivce) a fenotyp (pozorovatelné rysy vyplývající z interakce genotypu a prostředí). Obě teorie předpokládají, že naučené chování (zlepšení z lokálního hledání) ovlivňuje vývoj populace.

- **Baldwinovský přístup (Baldwin effect):** Naučené chování ovlivňuje mapování genotypů na fenotypy, což vede ke změnám ve fitness jednotlivců. Vylepšené řešení (fenotyp) se použije pouze k výpočtu fitness, ale samotné řešení (genotyp) se nemění a na potomky se přenáší původní genotyp. Tento přístup může pomoci vyhnout se lokálním optimům při konvergenci a zvyšuje diverzitu.
- **Lamarckovský přístup (Lamarckian evolution):** Předpokládá, že naučené chování ovlivňuje nejen fitness, ale přímo modifikuje genotyp jedince. Změněné vylepšené řešení (fenotyp) je dědičné a přímo se přenáší na potomky. Tento postup potenciálně urychluje hledání globálních minim.

4.2 Exaktní metody

I když jsou mnohé optimalizační problémy NP-těžké, instance relevantní v praxi mohou být dostatečně malé nebo mít strukturu vhodnou pro exaktní metody, které garantují nalezení optima. Tyto metody se navíc často kombinují s metaheuristikami (např. při křížení potomků).

- **Metody prohledávání stromu (Tree search, Branch and Bound):** Exaktní metody založené na rekurzivním rozdělování stavového prostoru do disjunktních podprostorů (branching), typicky omezováním domény proměnných, dokud nevzniknou jednotlivá řešení. Metody Branch and Bound (BnB) lze využít i uvnitř memetických algoritmů jako heuristický operátor křížení (např. v Dynastically Optimal Recombination) pro nalezení nejlepší kombinace rodičovských vlastností.
- **Dynamické programování:** Alternativní exaktní technika k prohledávání stavového prostoru, která je schopná i u velkých instancí efektivně najít optimální řešení.
- **Příklad - 0-1 Multiple Knapsack Problem (MKP):** Problém batohu ilustruje definice obou výše uvedených exaktních metod. Instance je dána vektorem zisků $V = \{v_0, \dots, v_{n-1}\}$, vektorem kapacit $C = \{c_0, \dots, c_{m-1}\}$ a maticí koeficientů kapacitních

omezení $M = \{m_{ij}\}$. Cílem je najít platný vektor B (kde $M \cdot B \leq C$) maximalizující účelovou funkci $m_P(y, x) = \sum_{i \in y} v_i$.

- **Programování s omezujícími podmínkami (Constraint programming - CP):** Algoritmy šíření omezení (Constraint propagation) poskytují způsob, jak časově efektivně a dramaticky snížit efektivní prohledávaný prostor vysoce omezeného problému. Pokud jsou tyto postupy začleněny do stromového vyhledávání (tree search), počet uzlů vyhledávacího stromu může být udržován natolik malý, že v praxi lze dobře vyřešit i velké instance problémů splňování omezení (CSPs).

5 Přednáška 3 - Neúplná řešení a dekodéry

5.1 Nepřímá a neúplná reprezentace řešení (Incomplete/indirect solution representations)

Nepřímá reprezentace řešení (známá např. z genetických algoritmů jako rozdíl mezi genotypem a fenotypem) se používá k transformaci složitého stavového prostoru, který může obsahovat těžko zvládnutelné omezující podmínky, na prostor, ve kterém lze snáze aplikovat standardní operátory místního prohledávání. V případě neúplné reprezentace operuje metaheuristika pouze nad podmnožinou rozhodovacích proměnných, zatímco chybějící části doplňuje optimálně (nebo dostatečně kvalitně) tzv. dekodér.

5.2 Metaheuristiky založené na dekodérech (Decoder-based metaheuristics)

Základní myšlenkou je využití "inteligentního" algoritmu uvnitř metaheuristiky. Dekodér provádí transformaci zakódovaného řešení a zajišťuje doplnění chybějících částí. Dekodérem může být libovolný algoritmus, který je dostatečně rychlý – od konstruktivních heuristik, exaktních procedur, mixed integer programování (MIP), až po dynamické (DP) či omezující (CP) programování.

5.3 Aplikace: Zobecněná minimální kostra (Generalized Minimum Spanning Tree - GMST)

Problém GMST je definován na neorientovaném úplném grafu s ohodnocenými hranami $G = (V, E, c)$, kde množina uzlů V je rozdělena do r po dvou disjunktních clusterů $V_1, \dots, V_r$. Cílem je najít graf bez cyklů $S = (P, T)$ (kostru) s minimální cenou hran, který obsahuje právě jeden uzel z každého clusteru.

5.3.1 Počáteční řešení (Initial solutions)

Pro rychlé získání počátečního řešení se používají dvě heuristiky (přičemž se pro následné prohledávání vybere to lepší):

- **Minimum Distance Heuristic (MDH):** Pro každý cluster vybere uzel s nejnižším součtem cen hran ke všem uzlům v ostatních clusterech. Na podgraf indukovaný těmito uzly se následně aplikuje Kruskalův algoritmus pro MST. Časová složitost je $O(|V|^2 + r^2 \log r)$.

- **Iterated Kruskal-Based Heuristic (IKH):** Hrany jsou zpracovávány vzestupně podle ceny. Hrana je do množiny přidána právě tehdy, když nevytvoří cyklus a zároveň z obou odpovídajících clusterů ještě nebyla dříve vybrána žádná jiná hrana. Proces se iteruje tak, že vždy zafixujeme jeden specifický uzel $u \in V$, který musí v řešení být. Časová složitost je $O(|V||E|)$.

5.3.2 NSB-VNS (Node set based VNS)

Řešení je v metaheuristice reprezentováno pouze výběrem uzlů (Node Set Based), tj. vektorem uzlů $p = (p_1, \dots, p_r) \in V_1 \times \dots \times V_r$. Dekodérem je v tomto případě Primův algoritmus pro MST s časovou složitostí $O(r^2)$, který doplní vhodné hrany.

- **NEN (Node Exchange Neighborhood):** Okolí obsahuje všechny vektory uzlů, které se od aktuálního liší v právě jednom uzlu (jeden uzel je nahrazen jiným uzlem ze stejného clusteru). Obsahuje $O(|V|)$ sousedů. Aby se urychlilo vyhodnocení celého okolí, využívá se *inkrementální vyhodnocení* (odstraněním uzlu vznikne několik komponent, ty se spojí s novým uzlem bez přepočítávání celého stromu).
- **R2NEN (Restricted Two-Nodes Exchange Neighborhood):** Protože obecná výměna k uzlů (k -NEN) je výpočetně příliš drahá ($O(|V|^k)$), využívá se omezená verze, kde se uvažuje současná výměna pouze takové dvojice uzlů, které jsou v aktuálním řešení spojeny hranou.
- **Průběh:** Algoritmus využívá Variable Neighborhood Descent (VND) kombinující prohledávání NEN a R2NEN, dokud nenajde lokální optimum. Shaking využívá náhodný tah z obecného k -NEN ($k = 3, \dots, k_{max}$).

Algorithm 15 Prozkoumávání okolí NEN (Výměna uzlu v NSB-VNS)

Vstup: Aktuální vektor uzlů $p = (p_1, \dots, p_r)$, aktuální kostra S

Výstup: Nejlepší nalezené sousední řešení

```

1:  $best\_solution \leftarrow S$ 
2: for all cluster  $V_i \in \{V_1, \dots, V_r\}$  do
3:   for all alternativní uzel  $p'_i \in V_i$  takový, že  $p'_i \neq p_i$  do
4:     Odstraň uzel  $p_i$  a všechny jeho incidentní hrany z aktuálního stromu  $S$ 
5:     // Pozorování: Strom se rozpadl na  $deg(p_i)$  odpojených komponent
6:     Přidej do grafu nový uzel  $p'_i$ 
7:     Pro každou odpojenou komponentu najdi nejkratší (nejlevnější) hranu, která ji
       spojuje s uzlem  $p'_i$ 
8:     Spoj komponenty s uzlem  $p'_i$  pomocí těchto nejkratších hran, čímž vznikne nový
       strom  $S'$ 
9:     Vypočítej účelovou hodnotu nového stromu  $S'$ 
10:    if  $cena(S') < cena(best\_solution)$  then
11:       $best\_solution \leftarrow S'$ 
12:    end if
13:    // Obnova původního stavu pro zkoušení dalšího uzlu
14:    Zruš propojení s  $p'_i$ , vrať původní uzel  $p_i$  a jeho hrany (obnov  $S$ )
15:  end for
16: end for
17: return  $best\_solution$ 

```

5.3.3 GESB-VNS (Global edge set based VNS)

Duální přístup k NSB. Řešení je zde neúplně reprezentováno jako strom složený z "globálních hran" T^g propojující jednotlivé clustery V_i (Global Edge Set) na globálním grafu G^g . Dekodérem je zde postup dynamického programování (DP), který dosazuje optimální volbu konkrétních uzlů.

- **Dynamické programování pro dekódování:** Globální strom S^g se zakoření v libovolném clusteru a prochází se metodou do hloubky (depth-first). Pro každý cluster V_i a uzel $v \in V_i$ se vypočítá minimální cena podstromu zakořeněného ve V_i , pokud v je vybraný uzel, podle rekurze:

$$c(T^g, V_i, v) = \begin{cases} 0 & \text{pokud } V_i \text{ je list v } S^g \\ \sum_{V_l \in \text{succ}(V_i)} \min_{u \in V_l} \{c(v, u) + c(T^g, V_l, u)\} & \text{jinak} \end{cases} \quad (1)$$

(kde $\text{succ}(V_i)$ jsou následníci V_i v S^g). V nejhorším případě běží v čase $O(|V|^2)$.

- **GEEN (Global Edge Exchange Neighborhood):** Okolí obsahuje všechny přípustné kostry, které se od globálního stromu T^g liší právě v jedné globální hraně. Odstranění hrany rozdělí strom na 2 komponenty, které se spojí jinou hranou ($O(r^3)$ sousedů). Ke zrychlení se využívá *Inkrementální DP*, kdy se rekurze aktualizuje jen na cestě od modifikovaných clusterů ke kořeni.

Algorithm 16 Exploration of GEEN

Vstup: řešení S

```

1: for all globální hrany  $(V_i, V_j) \in T^g$  do
2:   odstraň  $(V_i, V_j)$ 
3:    $M_1 \leftarrow$  seznam clusterů v komponentě  $K_1^g$  obsahující  $V_i$ , procházeno v preorder
4:    $M_2 \leftarrow$  seznam clusterů v komponentě  $K_2^g$  obsahující  $V_j$ , procházeno v preorder
5:   for all  $V_k \in M_1$  do
6:     ustav  $V_k$  jako kořen  $K_1^g$ 
7:     for all  $V_l \in M_2$  do
8:       ustav  $V_l$  jako kořen  $K_2^g$ 
9:       přidej  $(V_k, V_l)$ 
10:      použij inkrementální DP pro určení kompletního řešení a jeho účelové hodnoty
11:      if aktuální řešení je lepší než nejlepší then
12:        ulož aktuální řešení jako nejlepší
13:      end if
14:      odstraň  $(V_k, V_l)$ 
15:    end for
16:  end for
17: end for
18: return nejlepší řešení

```

5.3.4 Combined VNS (COMB-VNS)

Kombinovaný přístup využívající obě neúplné reprezentace současně a ukázal se jako nejúspěšnější.

- Vnitřní VND aplikuje místní prohledávání postupně na okolí **NEN**, **GEEN** a **R2NEN**. Toto pořadí je dáno jejich výpočetní složitostí (nejrychlejší se vyhodnocuje první).
- Shaking krok je prováděn přes oba typy reprezentací: k -tý shaking sestává z $k + 1$ náhodných tahů v NEN následovaných k náhodnými tahy v GEEN ($k = 2, \dots, k_{max}$). Tím se zajistí mnohem větší diverzifikace vyhledávání.

5.4 Algoritmické detaily inkrementálního vyhodnocování

Níže jsou formálně popsány procedury prozkoumávání okolí, které využívají inkrementální vyhodnocování (dočasné "rozbíjení" řešení) pro dramatické zrychlení výpočtu.

Algorithm 17 Prozkoumávání okolí GEEN (Výměna globální hrany v GESB-VNS)

Vstup: Aktuální globální strom T^g , kompletní řešení S s uzly dosazenými z DP

Výstup: Nejlepší nalezené sousední řešení

```

1:  $best\_solution \leftarrow S$ 
2: for all globální hranu  $(V_i, V_j)$  v aktuálním globálním stromu  $T^g$  do
3:   Odstraň hranu  $(V_i, V_j)$  z  $T^g$ 
4:   // Pozorování: Strom  $T^g$  se exaktně rozpadl na dvě komponenty  $K_1^g$  (obsahuje  $V_i$ )
   a  $K_2^g$  (obsahuje  $V_j$ )
5:   for all cluster  $V_k$  patřící do komponenty  $K_1^g$  do
6:     Dočasně ustav cluster  $V_k$  jako kořen podstromu  $K_1^g$ 
7:     for all cluster  $V_l$  patřící do komponenty  $K_2^g$  do
8:       Dočasně ustav cluster  $V_l$  jako kořen podstromu  $K_2^g$ 
9:       Propoj komponenty přidáním nové testovací globální hrany  $(V_k, V_l)$ 
10:      Aplikuj inkrementální Dynamické Programování (DP): přepočítej optimální
      volbu uzlů pouze na cestě od modifikovaných clusterů  $V_k$  a  $V_l$  nahoru ke kořeni
      nového stromu
11:      Nechť  $S'$  je takto nově vygenerované kompletní řešení
12:      if  $cena(S') < cena(best\_solution)$  then
13:         $best\_solution \leftarrow S'$ 
14:      end if
15:      Odstraň testovací hranu  $(V_k, V_l)$ 
16:    end for
17:  end for
18:  Vrať původní hranu  $(V_i, V_j)$  zpět do  $T^g$  (obnova původního stromu)
19: end for
20: return  $best\_solution$ 

```

6 Přednáška 4 - Redukce instancí problému (Problem Instance Reduction)

6.1 Algoritmus Construct, Merge, Solve & Adapt (CMSA)

Obecné přesné (exaktní) řešiče jako CPLEX nebo GUROBI jsou velmi efektivní a obsahují dekády zkušeností z operačního výzkumu, ale narazí, pokud je instance problému příliš

velká. Hlavní myšlenkou hybridní metaheuristiky CMSA je chytře zredukovat původní velkou instanci problému na mnohem menší pod-instanci (sub-instance), a tuto zredukovanou verzi následně předat exaktnímu řešiči. Pokud je redukce provedena chytře, sub-instance stále obsahuje vysoce kvalitní (nebo dokonce optimální) komponenty řešení.

Algoritmus iteruje přes 4 hlavní fáze:

- **Construct (Konstrukce):** Pravděpodobnostní generování sady platných řešení pomocí rychlé konstruktivní heuristiky.
- **Merge (Sloučení):** Komponenty (části) z vygenerovaných řešení se "slijí" (merge) do jedné omezené množiny C_{sub} , která představuje naši zredukovanou instanci problému.
- **Solve (Řešení):** Exaktní řešič (např. ILP solver) najde optimální řešení *pouze* za použití komponent z této omezené množiny C_{sub} .
- **Adapt (Adaptace):** Množina C_{sub} se dynamicky čistí od zbytečných komponent pomocí parametru "věku" (ageing), aby sub-instance v další iteraci neúměrně nenarostla a nezahltila řešič.

Algorithm 18 Construct, Merge, Solve & Adapt (CMSA)

Vstup: Instance problému, n_a (počet konstrukcí na iteraci), age_{max} (max. věk nepoužité komponenty)

Výstup: Nejlepší nalezené řešení s_{best}

```

1:  $C_{sub} \leftarrow \emptyset$  // Inicializace množiny komponent sub-instance
2:  $s_{best} \leftarrow \emptyset$ 
3: while podmínky ukončení nejsou splněny (např. časový limit) do
4:    $S_{cons} \leftarrow \emptyset$ 
5:   /// 1. Fáze: Construct
6:   for  $i = 1$  to  $n_a$  do
7:      $s \leftarrow ProbabilisticConstructiveHeuristic()$ 
8:      $S_{cons} \leftarrow S_{cons} \cup \{s\}$ 
9:   end for
10:   $\forall c \in C_{sub} : age(c) \leftarrow age(c) + 1$  // Zestárnutí všech dosavadních komponent
11:  /// 2. Fáze: Merge
12:   $C_{sub} \leftarrow C_{sub} \cup \{c \mid c \text{ je komponenta některého } s \in S_{cons}\}$ 
13:  /// 3. Fáze: Solve
14:   $s_{opt} \leftarrow ExactSolver(C_{sub})$  // Vyřešení zredukované instance pomocí exaktní metody
15:  if  $cena(s_{opt}) < cena(s_{best})$  then
16:     $s_{best} \leftarrow s_{opt}$ 
17:  end if
18:  /// 4. Fáze: Adapt
19:   $\forall c \in s_{opt} : age(c) \leftarrow 0$  // Resetování věku těch komponent, které řešič reálně využil
20:   $C_{sub} \leftarrow C_{sub} \setminus \{c \in C_{sub} \mid age(c) > age_{max}\}$  // Smazání starých a nepotřebných komponent
21: end while
22: return  $s_{best}$ 

```

6.2 Aplikace: Minimum Common String Partition (MCSP)

MCSP je NP-těžký problém kombinatorické optimalizace, nejčastěji se vyskytující v bio-informatice při porovnávání genových sekvencí. Na vstupu jsou dány dva řetězce s_1 a s_2 složené z konečné abecedy (musí obsahovat stejnou multimnožinu znaků).

Cílem je rozdělit s_1 na množinu nepřekrývajících se podřetězců P_1 a s_2 na množinu podřetězců P_2 tak, aby platilo $P_1 = P_2$ (tzn. obě množiny obsahují identické podřetězce), a zároveň byl celkový počet podřetězců $|P_1|$ **minimální** (co nejmenší).

6.2.1 Probabilistic solution generation (Pravděpodobnostní generování řešení)

Aby mohl CMSA fungovat, potřebuje získat základní platná řešení (fáze Construct). V případě MCSP to probíhá následovně:

- Algoritmus heuristicky s určitou dávkou náhody vybírá dosud nepokryté pozice v řetězcích.
- Snaží se najít shodu (stejný znak či blok znaků) mezi s_1 a s_2 a tento nalezený podřetězec se pokusí co nejvíce prodloužit.
- Vzniklý sdílený podřetězec se stane komponentou řešení a příslušné pozice v obou řetězcích se zablokují (označí jako pokryté).
- Pokud už nejde najít žádný delší blok, zbytek znaků se pokryje triviálním řešením (bloky o délce přesně 1 znak).
- Výsledné nalezené podřetězce se "slijí" (Merge) do globální množiny kandidátních komponent C_{sub} .

6.2.2 Solving reduced sub-instances (Řešení zredukovaných instancí)

Pokud bychom exaktnímu ILP modelu předali k řešení kompletní problém MCSP se stovkami tisíc možných párování všech podřetězců, výpočet by trval roky.

Trik CMSA spočívá v tom, že řešiči (např. CPLEXu) předáme ****výhradně**** podřetězce, které se zrovna teď nacházejí v naší omezené množině C_{sub} . Zredukovaný ILP model pak řeší mnohem jednodušší otázku: *"Jaký je matematicky nejmenší počet komponent z množiny C_{sub} , kterými dokážu beze zbytku pokrýt oba řetězce s_1 i s_2 ?"*.

Jelikož je C_{sub} relativně malá, řešič najde prověřené optimum pro tuto sub-instanci bleskově. Následná adaptace (Ageing) se pak postará o to, že podřetězce v C_{sub} , které řešič systematicky ignoruje, jsou smazány, aby udělaly místo novým podřetězcům v další iteraci.

7 Přednáška 5 - Hledání ve velkém okolí (Large Neighborhood Search - LNS)

7.1 Základní myšlenka a MIP-based LNS

Hlavní myšlenkou metody Large Neighborhood Search (LNS) je v každé iteraci vzít aktuálně platné (incumbent) řešení a **částečně jej destruovat** (odstranit určité komponenty), čímž vznikne neúplné, částečné řešení (tzv. partial solution $S_{partial}$). Fáze destrukce definuje ono "velké okolí".

Zatímco klasické lokální hledání by zkoušelo chybějící prvky doplnit jednoduchou heuristikou, MIP-based LNS na toto částečné řešení zavolá **exaktní MIP řešič** (Mixed Integer Programming solver, např. CPLEX). Řešič operuje nad původním ILP modelem problému, ke kterému se přidají omezující podmínky fixující proměnné z $S_{partial}$. Díky tomu, že je velká část proměnných zafixována, je problém výpočetně mnohem menší a MIP řešič je schopen v něm najít exaktní (nebo velmi kvalitní) optimum mnohem rychleji než u celého problému.

Algorithm 19 MIP-based Large Neighborhood Search (MIP-LNS)

Vstup: Instance problému I , časový limit t_{max} pro MIP řešič

```

1:  $S_{cur} \leftarrow GenerateInitialSolution()$ 
2:  $S_{bsf} \leftarrow S_{cur}$  //  $bsf = best\ so\ far$ 
3: while CPU time limit není dosažen do
4:    $S_{partial} \leftarrow DestroyPartially(S_{cur})$  // Destrukce: fixace proměnných
5:    $S'_{opt} \leftarrow ApplyExactSolver(S_{partial}, t_{max})$  // Oprava: exaktní dořešení zbytku
6:   if  $S'_{opt}$  je lepší než  $S_{bsf}$  then
7:      $S_{bsf} \leftarrow S'_{opt}$ 
8:   end if
9:    $S_{cur} \leftarrow ApplyAcceptanceCriterion(S'_{opt}, S_{cur})$  // Např. vždy přijmi lepší, nebo použij Simulated Annealing
10: end while
11: return  $S_{bsf}$ 

```

7.2 Aplikace 1: Problém minimální dominující množiny s vahami (MWDS)

Dán neorientovaný graf $G = (V, E)$ a pozitivní váha $w(v)$ pro každý uzel $v \in V$. Hledáme podmnožinu uzlů $S \subseteq V$ takovou, že každý uzel $v \in V$ buď patří do S , nebo má v S alespoň jednoho souseda. Cílem je minimalizovat sumu vah vybraných uzlů.

Základní ILP formulace:

$$\min \sum_{i=1}^n w(v_i) \cdot x_i \quad (2)$$

$$\text{s.t.} \quad \sum_{v_j \in N[v_i]} x_j \geq 1 \quad \forall v_i \in V \quad (3)$$

$$x_i \in \{0, 1\} \quad \forall i = 1, \dots, n \quad (4)$$

Kde $N[v_i]$ je uzavřené okolí (uzel + jeho sousedé). Rovnice (4.2) exaktně zajišťuje, že je každý uzel grafu pokryt.

7.2.1 Počáteční řešení (Greedy heuristika pro MWDS)

Konstruktivní heuristika postupně přidává uzly s nejlepším poměrem nově pokrytých sousedů vůči váze uzlu, dokud nejsou pokryty všechny uzly. Označme V_{cov} množinu dosud pokrytých uzlů. Počet dosud nepokrytých sousedů pro uzel v je definován jako $deg(v|V_{cov}) = |N(v) \cap V \setminus V_{cov}|$.

Algorithm 20 Greedy Heuristic for MWDS

Vstup: Graf $G = (V, E)$, váhy $w(v)$

```
1:  $S \leftarrow \emptyset, V_{cov} \leftarrow \emptyset$ 
2: while  $V_{cov} \neq V$  do
3:    $v^* \leftarrow \arg \max_{v \in V \setminus V_{cov}} \left\{ \frac{\deg(v|V_{cov})}{w(v)} \right\}$ 
4:    $S \leftarrow S \cup \{v^*\}$ 
5:    $V_{cov} \leftarrow V_{cov} \cup N[v^*]$ 
6: end while
7: return  $S$ 
```

7.2.2 Částečná destrukce a aplikace MIP (MIP-LNS pro MWDS)

Fáze destrukce se řídí podílem $perc_{dest}$, který určuje procento uzlů k odstranění z aktuálního řešení S_{cur} . Přesný počet odstraňovaných uzlů d je dán jako $d = \max \left\{ 3, \left\lfloor \frac{perc_{dest} \cdot |S_{cur}|}{100} \right\rfloor \right\}$.

Uzly k odstranění se volí dvěma strategiemi:

- $type_{dest} = 0$: Uzly se odstraňují zcela rovnoměrně náhodně.
- $type_{dest} = 1$: Uzly se odstraňují pravděpodobnostně se zaujetím (heuristically biased). Pravděpodobnost odstranění $p(v)$ je úměrná nevýhodnosti uzlu (velká váha a malý stupeň):

$$p(v) = \frac{w(v)/\deg(v)}{\sum_{v' \in S_{cur}} w(v')/\deg(v')} \quad \forall v \in S_{cur} \quad (5)$$

Hodnota $perc_{dest}$ se v průběhu iterací dynamicky adaptuje. Pokud algoritmus najde zlepšení, $perc_{dest}$ se resetuje na spodní hranici $perc_{dest}^l$. Pokud se nezlepší, zvýší se o 5 %, aby algoritmus ve zhoršujících se případech v dalším kroku destruoval větší část řešení a prohledal tak širší okolí.

Aplikace řešiče: Necht $S_{partial}$ je množina uzlů, která po destrukci v řešení zbyla. MIP řešič pak řeší původní ILP pro MWDS obohacené o podmínku $x_i = 1 \quad \forall v_i \in S_{partial}$.

7.3 Aplikace 2: Zobecněná minimální kostra (GMST)

V minulé přednášce (přednáška 3) jsme definovali algoritmus COMB-VNS pracující s neúplnými reprezentacemi. Zde je do něj přidáno další "velké" okolí, konkrétně **Global Subtree Optimization Neighborhood (GSON)**.

Princip GSON je následující:

- Uvažujeme aktuální řešení $S = (P, T)$ a jeho globální strom $S^g = (V^g, T^g)$.
- S^g se zakoření v náhodném clusteru V_{root} a pomocí prohledávání do hloubky (DFS) se určí všechny platné podstromy $Q_1, \dots, Q_k$, které obsahují minimálně N_{min} a maximálně N_{max} clusterů.
- V každé iteraci velkého okolí je vybraný podstrom Q_i oddělen od hlavního stromu S^g .
- Uzly a clusterly obsažené v podstromu Q_i jsou vnímány jako zcela izolovaný, samostatný menší problém GMST, který se exaktně vyřeší pomocí lokálně-globálního MIP modelu, čímž získáme nově provázaný, plně optimalizovaný podstrom Q'_i .

- Následně se Q'_i připojí zpět ke zbytku řešení tím nejlepším možným způsobem (podobně jako v okolí GEEN z 3. přednášky).

Algorithm 21 Exploration of GSON

Vstup: Aktuální řešení S , parametry N_{min}, N_{max}

- 1: Urči kořeny $V_1, \dots, V_k$ příslušných podstromů $Q_1, \dots, Q_k$ splňujících omezení N_{min} a N_{max} na počet clusterů
 - 2: **for** $i = 1 \dots k$ **do**
 - 3: Odstraň hranu spojující V_i s jejím rodičem (čímž se Q_i izoluje od S)
 - 4: Optimalizuj podstrom Q_i zavoláním exaktního MIP řešiče
 - 5: Připoj vylepšený podstrom Q_i zpět do S nejlepším možným způsobem (reconnection)
 - 6: **if** aktuální řešení je levnější než dosud nejlepší **then**
 - 7: Ulož jej jako nejlepší nalezené
 - 8: **end if**
 - 9: Obnov původní stav (vrať se zpět k výchozímu S před odříznutím) pro zkoušení dalšího podstromu
 - 10: **end for**
 - 11: **return** nejlepší nalezené řešení
-

8 Přednáška 6 - Paralelní a nezávislá konstrukce řešení (Parallel Non-independent Construction)

8.1 Obecná myšlenka (General idea)

Většina metaheuristik (jako např. Ant Colony Optimization - ACO nebo GRASP) konstruuje řešení sekvenčně a zcela nezávisle na sobě. Procházejí vyhledávací prostor probabilisticky a využívají pouze tzv. *primární znalost problému* (heuristickou/chamtivou informaci).

Na druhou stranu, klasické metody prohledávání stromu (jako Branch-and-Bound nebo Beam Search) využívají navíc *komplementární znalost problému* – informace o mezích (dolní nebo horní meze, tzv. bounds). Tyto meze slouží k prořezávání větví vyhledávacího stromu a k rozlišování kvality částečných řešení na stejné úrovni stromu. Tyto metody tak konstruují řešení (pseudo-)paralelním a vzájemně závislým (non-independent) způsobem. Cílem této kapitoly je obohatit metaheuristiky o tyto vlastnosti.

8.2 Beam Search

Beam Search je deterministická metoda (neúplná varianta Branch-and-Bound), která prohledává strom řešení do šířky (breadth-first search).

- Na každé úrovni stromu udržuje množinu částečných řešení B , zvanou **paprsek (beam)**.
- Každé částečné řešení v B je rozšířeno nanejvýš k_{ext} způsoby na základě chamtivé funkce (greedy weight), čímž vznikne nová množina částečných řešení B' .

- Množina B' je následně zredukována na maximálně b_{width} řešení (tzv. **beam width** - šířka paprsku). Tato redukce probíhá na základě ohodnocení pomocí omezující funkce (upper bound UB pro maximalizaci nebo lower bound LB pro minimalizaci).

Algorithm 22 Beam Search (pro maximalizaci)

Vstup: Instance problému I , parametry k_{ext} a b_{width}

```

1:  $S_{partial} \leftarrow \emptyset$ ,  $B \leftarrow \{S_{partial}\}$ ,  $S_{bsf} \leftarrow \text{NULL}$ 
2: while  $B$  není prázdná do
3:    $B' \leftarrow \emptyset$ 
4:   for all  $S_{partial} \in B$  do
5:     if  $S_{partial}$  je kompletní řešení then
6:       Aktualizuj nejlepší nalezené řešení  $S_{bsf}$ 
7:     else
8:       Vyber nanejvýš  $k_{ext}$  nejlepších rozšíření podle heuristické funkce  $g$  a přidej je do  $B'$ 
9:     end if
10:  end for
11:   $B \leftarrow$  vyber nejlepších  $\min\{|B'|, b_{width}\}$  částečných řešení z  $B'$  podle jejich horní meze  $UB()$ 
12: end while
13: return  $S_{bsf}$ 

```

8.3 Beam-ACO: Beam search + Ant Colony Optimization

Beam-ACO je hybridní metaheuristika, která spojuje sílu ACO s algoritmem Beam Search. Její celkový rámec zůstává jako u ACO (využívá feromony, dělá se update atd.), ale s jedním zásadním rozdílem: **Místo nezávislé konstrukce n_a řešení v každé iteraci mravenci aplikují pravděpodobnostní Beam Search.**

Tento pravděpodobnostní Beam Search nevybírá rozšíření řešení čistě deterministicky, ale probabilisticky – na základě kombinace chamtivé heuristické informace a uložených feromonových hodnot.

8.4 Aplikace: Vícerozměrný problém batohu (Multidimensional Knapsack Problem - MKP)

MKP je NP-těžký problém. Je dána množina položek $I = \{1, \dots, n\}$ a množina zdrojů $K = \{1, \dots, m\}$. Každý zdroj k má kapacitu c_k a každá položka i vyžaduje spotřebu $r_{i,k}$ od zdroje k . Každá položka má také zisk p_i . Cílem je vybrat podmnožinu položek tak, aby maximalizovala celkový zisk a nepřekročila kapacity zdrojů.

ILP model využívá binární proměnné $x_i \in \{0, 1\}$: $\max \sum_{i=1}^n p_i x_i$ za podmínek $\sum_{i=1}^n r_{i,k} x_i \leq c_k$ pro všechna k .

8.4.1 Chamtivá heuristika (Greedy heuristic)

Heuristika předpokládá, že položky jsou seřazeny sestupně podle jejich užitkové hodnoty (utility) u_i , která je definována jako poměr zisku a celkové relativní spotřeby zdrojů:

$$u_i = \frac{p_i}{\sum_{k=1}^m r_{i,k}/c_k} \quad i = 1, \dots, n \quad (6)$$

Algoritmus (Constructive Heuristic) prochází seřazené položky a přidává je do řešení S , dokud se vejdu do zbývajících kapacit všech zdrojů.

8.4.2 Beam search pro MKP

V rámci deterministického Beam Search je potřeba kvalitní omezující funkce $UB(S_{partial})$ pro redukci paprsku na velikost b_{width} . Jako $UB(S_{partial})$ se pro MKP využívá **Lineární programovací (LP) relaxace** původního ILP modelu řešená exaktně (např. CPLEXem). Relaxace spočívá v tom, že položkám již vybraným do $S_{partial}$ se proměnná fixuje na $x_i = 1$, zatímco doména zbývajících proměnných se uvolní na reálná čísla $0 \leq x_i \leq 1$. Bylo experimentálně ověřeno, že tato LP relaxace má obrovskou *diskriminační sílu* a spolehlivě dokáže určit, které částečné řešení povede k lepšímu konečnému výsledku.

8.4.3 Čistý ACO přístup (pure-ACO)

Čistý ACO implementuje variantu *MAX-MIN Ant System (MMAS)* v rámci *Hyper-Cube Frameworku (HCF)*.

- Feromony $\tau_i \geq 0$ jsou uloženy na položkách a jsou inicializovány na 0.5.
- Konstrukce řešení vychází z chamtivé heuristiky, ale s parametrem d_{rate} určujícím míru determinismu. V každém kroku se vytvoří množina L kandidátů o velikosti l_{size} . S pravděpodobností d_{rate} se vybere nejlepší, jinak se volí z L náhodně.
- Update feromonů probíhá pomocí tří řešení: iteration-best (S_{ib}), restart-best (S_{rb}) a best-so-far (S_{bsf}), přičemž jejich váha závisí na dynamicky počítaném *konvergenčním faktoru* cf .

8.4.4 Beam-ACO pro MKP

Jak již bylo naznačeno, Beam-ACO přejímá rámec čistého ACO (včetně updatu feromonů, S_{ib} , S_{rb} a S_{bsf}), ale nezávislou konstrukci řešení nahrazuje. Fáze konstrukce se změní tak, že se zavolá algoritmus Beam Search z předchozí podkapitoly, ovšem výběr k_{ext} rozšíření do množiny B' již není exaktní (deterministický) podle heuristiky, ale probíhá probabilisticky jako u ACO, přičemž využívá jak užitkovou funkci u_i , tak hodnotu feromonů τ_i .

8.5 Zastřešující ACO algoritmus pro MKP

Tento algoritmus představuje hlavní smyčku čistého ACO přístupu (pure-ACO) pro vícerozměrný problém batohu (MKP). Využívá tři referenční řešení pro aktualizaci feromonů: nejlepší v iteraci (S_{ib}), nejlepší od posledního restartu (S_{rb}) a absolutně nejlepší dosud nalezené (S_{bsf}).

Zároveň obsahuje mechanismus pro restart feromonů. Míru konvergence měří tzv. konvergenční faktor $cf \in [0, 1]$. Pokud se feromony příliš usadí ($cf > 0.99$), algoritmus nejprve zkusí změnit strategii updatu (přepne **bs_update**), a pokud to nepomůže, feromony kompletně vyresetuje.

Algorithm 23 ACO pro MKP (MAX-MIN Ant System)

Vstup: Instance problému MKP, parametry d_{rate}, l_{size}, n_a (počet mravenců)

```
1:  $S_{bsf} \leftarrow \emptyset, S_{rb} \leftarrow \emptyset$ 
2:  $cf \leftarrow 0, bs\_update \leftarrow \text{FALSE}$ 
3:  $\tau_i \leftarrow 0.5$  pro všechny feromony  $\tau_i \in \Upsilon$  // Inicializace feromonů na neutrální střední hodnotu
4: while CPU time limit není dosažen do
5:   /// 1. Fáze: Konstrukce řešení mravenci
6:   for  $cnt = 1$  to  $n_a$  do
7:      $S_{cnt} \leftarrow \text{ConstructSolution}()$  // Zde se volá algoritmus z předchozí podkapitoly
8:   end for
9:   /// 2. Fáze: Vyhodnocení a aktualizace nejlepších řešení
10:   $S_{ib} \leftarrow \arg \max\{f(S_{cnt}) \mid cnt = 1, \dots, n_a\}$  // Nejlepší řešení v aktuální iteraci
11:  if  $f(S_{ib}) > f(S_{rb})$  then
12:     $S_{rb} \leftarrow S_{ib}$  // Aktualizace nejlepšího řešení od posledního restartu
13:  end if
14:  if  $f(S_{ib}) > f(S_{bsf})$  then
15:     $S_{bsf} \leftarrow S_{ib}$  // Aktualizace globálně nejlepšího řešení
16:  end if
17:  /// 3. Fáze: Aktualizace feromonů a konvergence
18:   $\text{ApplyPheromoneUpdate}(cf, bs\_update, \Upsilon, S_{ib}, S_{rb}, S_{bsf})$  // Přidá feromony na položky z nejlepších řešení a vypaří ostatní
19:   $cf \leftarrow \text{ComputeConvergenceFactor}()$  // Spočítá, jak moc jsou feromony "ustálené"
20:  /// 4. Fáze: Zpracování uvíznutí (Restart mechanismus)
21:  if  $cf > 0.99$  then
22:    if  $bs\_update == \text{TRUE}$  then
23:       $\tau_i \leftarrow 0.5$  pro všechny feromony  $\tau_i \in \Upsilon$  // Tvrdý restart feromonů
24:       $S_{rb} \leftarrow \text{NULL}$ 
25:       $bs\_update \leftarrow \text{FALSE}$ 
26:    else
27:       $bs\_update \leftarrow \text{TRUE}$  // Změna strategie (příprava na restart)
28:    end if
29:  end if
30: end while
31: return  $S_{bsf}$  // Vrácení globálně nejlepšího řešení
```

9 Přednáška 7 - Konstrukce řešení a Constraint Programming (CP)

9.1 Hybridizace konstruktivních metaheuristik a Constraint Programming

Při řešení těžkých kombinatorických problémů často narazíme na dva protichůdné cíle: nalezení *platného* řešení (splnění všech tvrdých omezení) a nalezení *optimálního* řešení (minimalizace/maximalizace účelové funkce).

Základní myšlenka této hybridizace staví na komplementárních vlastnostech dvou přístupů:

- **Constraint Programming (CP):** Exceluje v prořezávání vyhledávacího prostoru (tzv. *domain filtering* a *constraint propagation*). Dokáže rychle detekovat slepé uličky a zajistit, aby algoritmus neporušil tvrdá omezení. Nevýhodou CP je ale slabá schopnost optimalizace (postrádá silnou globální vizi, jak najít to *nejlepší* z platných řešení).
- **Konstruktivní metaheuristiky (např. ACO, GRASP):** Excelují v optimalizaci díky učení a globální paměti (feromony). Skvěle navigují vyhledávacím prostorem směrem k optimu. Jejich slabinou je ale "slepota" vůči složitým tvrdým omezením – mravenci často zbytečně plýtvají časem generováním neplatných řešení, která pak musí penalizovat nebo zahazovat.

Spojením vzniká hybrid (často označovaný jako **CP-based ACO**), kde CP slouží jako "ochranný mantinel", který v každém kroku omezí možnosti, a ACO slouží jako "mozek", který z těchto povolených možností vybere tu nejlepší.

9.2 Integrace Ant Colony Optimization (ACO) a Constraint Programming (CP)

Integrace probíhá na velmi nízké úrovni (*low-level, tight coupling*). Mravenec nestaví řešení sám, ale v každém kroku se dotazuje CP modulu, jaké tahy jsou bezpečné.

Proces probíhá následovně:

1. **Inicializace:** Vytvoří se CP model s proměnnými a jejich doménami (všemi možnými hodnotami). Spustí se prvotní propagace omezení, která rovnou smaže hodnoty porušující pravidla.
2. **Dotaz na CP (Krok mravence):** Než mravenec vybere další komponentu (např. další uzel nebo úlohu), podívá se na aktuální zredukovanou doménu dané proměnné.
3. **Pravděpodobnostní výběr:** Mravenec aplikuje standardní ACO pravidlo (kombinace feromonů τ a heuristiky η) **výhradně na hodnoty, které prošly přes CP filtr.**
4. **Aplikace tahu a Propagace:** Mravenec provede volbu. Tato volba se předá do CP modulu jako nové omezení (proměnná = hodnota). CP modul okamžitě proběhne zbytek domén ostatních proměnných a prořeže je (odstraní hodnoty, které by v budoucnu vedly ke konfliktu s tímto novým tahem).

Algorithm 24 ACO + CP Integrace (Konstrukce řešení mravencem)

Vstup: Inicializovaný CP model, Feromonová matice τ

```
1:  $S \leftarrow \emptyset$  // Prázdné řešení
2:  $CP.Propagate()$  // Úvodní prořezání domén
3: while řešení  $S$  není kompletní do
4:    $X_i \leftarrow$  vyber další proměnnou k přiřazení
5:    $D(X_i) \leftarrow CP.GetDomain(X_i)$  // Získej pouze platné hodnoty z CP
6:   if  $D(X_i)$  je prázdná then
7:     Odmítnutí: Zde selhala konstrukce (slepá ulička, nutný backtrack)
8:     return Infeasible
9:   end if
10:  // Pravděpodobnostní výběr mravence pouze z platných možností
11:   $v^* \leftarrow \text{ProbabilisticChoice}(D(X_i), \tau, \eta)$ 
12:   $S \leftarrow S \cup \{(X_i = v^*)\}$ 
13:   $CP.AssignAndPropagate(X_i = v^*)$  // CP zafixuje hodnotu a prořeže zbytek pro-
    storu
14: end while
15: return  $S$ 
```

9.3 Aplikace: Rozvrhování strojů se sekvenčně závislými časy přípravy (Machine Scheduling with Sequence-dependent Setup Times)

Tato hybridní metoda je ideální pro silně omezené rozvrhovací problémy, jako je rozvrhování na jednom stroji (Single Machine Scheduling), kde čas potřebný na přípravu (setup time s_{ij}) silně závisí na tom, jaká úloha i předcházela úloze j (např. čištění stroje při změně černé barvy na bílou trvá déle než z bílé na černou).

Kromě sekvenčně závislých časů problém obvykle obsahuje ****tvrdá časová okna****:

- r_j (release date): Úloha nesmí začít dříve než v tento čas.
- d_j (due date / deadline): Úloha musí být dokončena nejpozději do tohoto času.

Jak zde funguje CP + ACO:

- **Role CP (Tvrdá omezení):** CP model si udržuje nejčasnější možný čas začátku (EST) a nejpozdější možný čas začátku (LST) pro každou dosud nezařazenou úlohu. Když mravenec zařadí úlohu A , CP model okamžitě připočítá čas běhu úlohy A a časy přípravy $s_{A,j}$ ke všem zbývajícím úlohám j . Pokud u nějaké úlohy zjistí, že by už nestihla svůj deadline (tzv. $EST_j > LST_j$), CP modul okamžitě zakáže zařadit úlohu A a odstraní ji z aktuální domény mravence.
- **Role ACO (Optimalizace):** Ze všech úloh, které aktuálně lze naplánovat tak, abychom neporušili žádný deadline (což mravenci řekne CP), si mravenec vybírá podle toho, kde jsou nejsilnější feromony (která sekvence se z dlouhodobého hlediska kolonie nejvíce vyplatí) a podle heuristiky (např. pravidlo nejbližšího deadline nebo nejkratšího času přípravy).

Výsledek? Mravenci nestráví 90 % času generováním rozvrhů, které jsou nepoužitelné kvůli porušení termínů. Staví pouze platné rozvrhy a plně se soustředí na minimalizaci celkového času přestaveb strojů.

10 Přednáška 8 - Hybridizace založená na kompletních archivech řešení (Complete Solution Archives)

10.1 Základní myšlenka

Většina metaheuristik (jako např. evoluční algoritmy) sdílí společnou slabinu: v průběhu hledání často opakovaně generují, navštěvují a vyhodnocují naprosto stejná kandidátní řešení. Toto chování plýtvá cenným výpočetním časem, zvláště v pozdějších fázích hledání (kdy konverguje a ztrácí diverzitu) a u problémů, kde je samotné ohodnocení řešení (fitness) výpočetně velmi drahé.

Základní myšlenkou je proto rozšířit metaheuristiku o paměťově efektivní **kompletní archiv řešení (Complete Solution Archive)**, který:

- Ukládá všechna kandidátní řešení, která algoritmus dosud zvažoval.
- Umožňuje extrémně rychlou kontrolu, zda je nově vygenerované řešení duplikát (zda už v archivu je).
- Poskytuje efektivní funkci pro **transformaci** duplicitního řešení na garantovaně nové, doposud neprozkoumané řešení, které je ale tomu původnímu velmi podobné (tzv. informovaná mutace).

10.2 Archiv řešení založený na struktuře Trie

Pro efektivní implementaci archivu se nevyužívají standardní hashovací tabulky (ty neumí efektivní transformaci), ale datová struktura zvaná **Trie** (prefixový strom).

Předpokládejme problém s binární reprezentací o délce l (prostor $\{0,1\}^l$). Trie je binární strom o maximální výšce $l - 1$, kde i -tá úroveň stromu odpovídá proměnné x_i . Každý uzel má dva ukazatele (pro $x_i = 0$ a $x_i = 1$). Ukazatel může odkazovat na další uzel, nebo obsahovat jeden ze dvou speciálních příznaků:

- **empty (prázdný)**: V tomto podprostoru zatím nebylo navštíveno žádné řešení.
- **completed (kompletní)**: Všechna možná řešení v tomto podprostoru již algoritmus navštívil.

Aby strom zbytečně nerostl, uplatňuje se **pruning (prořezávání)**: jakmile mají obě větve daného uzlu příznak *completed*, celý uzel se smaže a jeho rodiči se do dané větve zapíše *completed*.

10.2.1 Transformace řešení (Informed Mutation)

Když evoluční algoritmus vygeneruje řešení x , které už v archivu existuje (dojdeme po cestě až na příznak *completed*), musíme ho transformovat na řešení x' . Algoritmus se vrací po stromu nahoru k uzlu, který má alespoň jednu větev bez příznaku *completed* (tzv. bod odklonu). Zde proměnnou překloupí ($x_i = 1 - x_i$) a následně jde dolů, přičemž se snaží zachovat co nejvíce původních hodnot z x , aby bylo nové řešení x' co nejpodobnější.

Existují dvě hlavní strategie pro výběr bodu odklonu:

- **Deepest Position Transformation (DPT)**: Vždy vybere nejhlubší možný bod odklonu. Má nevýhodu (bias) v tom, že mění převážně proměnné s vysokými indexy na konci řetězce.

- **Random Position Transformation (RPT):** Bod odklonu vybere zcela náhodně ze všech dostupných na cestě, čímž eliminuje tento bias.

Algorithm 25 Transformace řešení (Transform Solution v Trie)

Vstup: Duplicitní řešení x , kořen Trie $root$

Výstup: Garantovaně nové (dosud nenavštívené) řešení x'

```

1: Najdi  $x$  v Trie a ulož si všechny možné body odklonu (kde druhá větev  $\neq completed$ )
   do seznamu devpoints
2:  $(i, p) \leftarrow$  vyber bod odklonu z devpoints (dle strategie DPT nebo RPT)
3:  $x_i \leftarrow 1 - x_i$  // Provedení odklonu (mutace)
4: while  $i \leq l$  do
5:   if  $p.next[x_i] == completed$  then
6:      $x_i \leftarrow 1 - x_i$  // Původní hodnota nelze použít, musíme vzít tu druhou
7:   end if
8:   if  $p.next[x_i] == empty$  then
9:     Vytvoř nový uzel a napoj ho
10:  end if
11:   $p \leftarrow p.next[x_i]$ 
12:   $i \leftarrow i + 1$ 
13: end while
14: Označ nový list jako completed a proved' prořezání stromu zespodu nahoru (pruning)
15: return  $x'$ 

```

10.3 Aplikace na GMST a prořezávání (Bounding)

Archiv řešení lze aplikovat i na složitější reprezentace, jako je NSB (vektor uzlů) a GESB (globální hrany) u problému Generalized Minimum Spanning Tree (GMST). Zatímco u binárního stromu měl každý uzel jen dvě větve, u NSB stromu má tolik větví, kolik je uzlů v daném clusteru.

Zásadním a extrémně silným vylepšením Trie archivu je však přidání konceptu **určování mezí (Bounding)**, známého z metody Branch-and-Bound.

10.3.1 Pruning the Trie by Bounding

Kromě odstraňování uzlů, které jsou plně prozkoumané, můžeme prořezávat i uzly, u kterých *teoreticky* dokážeme, že se v nich neskrývá žádné dobré řešení:

1. Pro částečné řešení (odpovídající danému uzlu v Trie) algoritmus heuristicky spočítá **dolní mez (Lower Bound)**.
2. V případě NSB reprezentace se dolní mez počítá tak, že se zafixované clusteru spojí přesně určenými uzly, zatímco u volných (nezafixovaných) clusterů se povolí spojení jakýmkoliv nejlevnějším uzlem (relaxace problému) a na to celé se pustí algoritmus pro minimální kostru (MST).
3. Pokud je tato vypočtená dolní mez **horší nebo rovna** objektivní hodnotě dosud nejlepšího nalezeného řešení (S_{bsf}), znamená to, že i ten nejlepší možný výsledek v tomto podstromu by byl k ničemu.

4. Algoritmus proto do tohoto uzlu zapíše příznak *completed* (čímž celou obrovskou část prostoru nenávratně zahodí a zmenší tak prohledávaný prostor), aniž by ho reálně musel prohledávat.

Tato integrace metaheuristiky (Evoluční algoritmus), kompletního archivu řešení a exaktního prořezávání (Bounding) ukazuje další silnou úroveň hybridizace, která dramaticky zrychluje proces u výpočetně náročných instancí.

11 Přednáška 9 - Hyper-heuristiky (Hyper-heuristics)

11.1 Základní koncept a motivace

V praxi (např. při dynamickém plánování rozvozů nebo rozvrhování) jsou exaktní metody často výpočetně neúnosné a tradiční heuristiky mívají specifické slabiny, kvůli kterým mohou v určitých situacích selhat. Navíc nalezení absolutního optima často nedává smysl, protože problém se v reálném čase mění a optimum může být velmi křehké.

Zatímco klasické metaheuristiky prohledávají prostor řešení, **hyper-heuristiky prohledávají prostor heuristik**. Jsou definovány jako "heuristiky pro výběr heuristik" nebo přesněji jako automatizované metodologie pro výběr nebo generování heuristik k řešení těžkých výpočetních problémů.

Hyper-heuristiky se dělí na dva hlavní směry:

1. **Výběr heuristik (Heuristic Selection):** Algoritmus zkoumá, jak chytře kombinovat a přepínat mezi již existujícími (nízkourovňovými) heuristikami tak, aby se vyrušily jejich individuální slabiny.
2. **Generování heuristik (Heuristic Generation):** Algoritmus (často pomocí Genetického programování) automaticky objevuje a skládá zcela nové heuristiky ze základních stavebních bloků.

11.2 Aplikace 1: Výběrové hyper-heuristiky pro rozvrhování (Timetabling)

Problémy jako tvorba univerzitních rozvrhů (Timetabling) lze modelovat jako **barvení grafu (Graph-coloring)**. Události (přednášky) jsou uzly, a pokud dvě události nesmí probíhat současně, vede mezi nimi hrana. Cílem je přiřadit uzlům barvy (časová okna).

Místo jedné heuristiky dáváme hyper-heuristice k dispozici sadu jednoduchých konstruktivních (nízkourovňových) heuristik, které určují, jaký uzel obarvit dál:

- **LD (Largest Degree):** Vybere událost s největším počtem tvrdých omezení.
- **LWD (Largest Weighted Degree):** Jako LD, ale omezení mají různé váhy.
- **CD (Color Degree):** Vybere událost, která koliduje s nejvíce již naplánovanými událostmi.
- **LE (Largest Enrolment):** Vybere událost s největším počtem účastníků.
- **SD (Saturation Degree):** Vybere událost, které zbývá nejméně volných časových oken.

- **RO (Random Order):** Vybere událost zcela náhodně.

Vysokoúrovňový řídicí algoritmus (např. Tabu Search) následně hledá **optimální sekvenci** těchto heuristik (např. sekvenci *LD-SD-SD-LE...*), kdy každá heuristika v sekvenci umístí přesně dvě události a předá řízení další. Pokud sekvence vytvoří neplatný rozvrh, její problémová pod-sequvence se umístí na Tabu list.

Algorithm 26 Výběrová hyper-heuristika (Selection Hyper-heuristic)

Vstup: Sada nízkoúrovňových heuristik $H = \{h_1, h_2, \dots, h_k\}$, instance problému

Výstup: Nejlepší nalezené řešení S_{best}

```

1: Inicializuj řídicí algoritmus (např. Tabu Search) a vygeneruj počáteční sekvenci heuristik seq
2:  $S_{best} \leftarrow \text{NULL}$ 
3: while není dosaženo kritéria ukončení do
4:    $S \leftarrow \emptyset$  // Prázdné částečné řešení
5:   for all heuristice  $h_i$  v sekvenci seq do
6:      $S \leftarrow$  Aplikuj  $h_i$  na  $S$  pro zařazení dalšího prvku
7:   end for
8:    $S \leftarrow$  Aplikuj lokální prohledávání na  $S$  pro finální vylepšení
9:   if  $\text{Cena}(S) < \text{Cena}(S_{best})$  then
10:     $S_{best} \leftarrow S$ 
11:   end if
12:   seq  $\leftarrow$  Uprav sekvenci pomocí Tabu Search (změň pořadí nebo typ heuristik)
13: end while
14: return  $S_{best}$ 

```

11.3 Aplikace 2: Generativní hyper-heuristiky pro splnitelnost (Boolean SAT)

Problém Boolean Satisfiability (SAT) hledá ohodnocení logických proměnných tak, aby byla splněna logická formule. Existují fixní heuristiky jako *GSAT* (překlop proměnnou, která splní nejvíce klauzulí) nebo *WalkSat*.

Generativní hyper-heuristika používá **Genetické programování (GP)**, aby tyto pevné přístupy nahradila evolucí. GP strom reprezentuje vlastní program (novou heuristiku). Netrénují se konkrétní řešení, ale **rozhodovací stromy**, které pracují s funkcemi jako:

- **FLIP(*v*)**: Překlopí hodnotu zadané proměnné.
- **MAX_SCR(*list*)**: Vráti proměnnou ze seznamu, která po překlopení získá nejvyšší počet splněných klauzulí (chamtivý výběr).
- **IFV(*prob*, *v1*, *v2*)**: S pravděpodobností *prob* vrátí proměnnou *v1*, jinak *v2* (stochastický prvek).

Jedinci v populaci (různé stromy/heuristiky) jsou hodnoceni podle toho, kolik cvičných problémů dokázali úspěšně vyřešit a kolik tahů k tomu potřebovali. Výsledná vygenerovaná heuristika se pak nasadí na zcela nové, neviděné instance.

11.4 Teoretické ukotvení: No Free Lunch (NFL) teorém

Zásadní teoretickou otázkou je: Může hyper-heuristika najít algoritmus, který je absolutně "nejlepší"? Odpověď dává slavný **No Free Lunch (NFL) teorém** (Wolpert, 1995). Ten matematicky dokazuje, že zprůměrujeme-li výkon přes *naprosto všechny* představitelné problémy, mají všechny optimalizační algoritmy přesně stejnou efektivitu.

Znamená to, že se snažíme zbytečně? Ne. Schumacher et al. ukázali, že NFL teorém platí pouze pro množiny problémů, které jsou uzavřené vůči permutaci hodnot (tedy pro zcela náhodné "šumové" problémy bez jakékoliv struktury). Reálné problémy (jako TSP, SAT nebo balení batohu) v sobě mají vysokou míru struktury a nejsou náhodné. Hyper-heuristiky tedy dávají obrovský smysl, protože nehledají jeden univerzální algoritmus pro všechno, ale učí se najít algoritmus perfektně ušitý na míru **specifické struktury** dané rodiny problémů.